

Spec-Driven Development · Taller práctico

Del enunciado ambiguo a la especificación ejecutable

Emilio Delgado Muñoz

Institutos Universitarios de Tecnologías Informáticas Aplicadas
Universidad de Extremadura · edm@unex.es

Extremadura Digital Day (EDD) · 2026

utilizando Spec Kit v1.0.2 y Claude Code vía OpenRouter

Antes de empezar: qué NO vamos a hacer hoy

- **No vamos a dar una charla de teoría.** Cinco minutos de marco y al teclado.
- **No vamos a terminar una aplicación.** Es imposible en 90 minutos y no es el objetivo.
- **No vamos a medir** cuánto código genera el agente.

Lo que sí vamos a hacer

Ejecutar el ciclo completo sobre un problema pequeño y comprobar, con las manos, dónde está la diferencia real entre esto y pedirle código a un agente sin más.

Objetivos

Al terminar deberíais poder:

1. Inicializar un proyecto con Spec Kit y explicar qué hay dentro de `.specify/`.
2. Escribir una constitución con principios *verificables*.
3. Revisar críticamente una especificación generada: detectar decisiones que tomó el agente y vosotros no.
4. Resolver ambigüedades de forma explícita.
5. Derivar plan y tareas, y detectar incoherencias entre artefactos.
6. Absorber un cambio de requisito modificando la especificación, no el código.

La idea que sostiene todo el taller

*El artefacto valioso es la especificación.
El código es una consecuencia regenerable de ella.*

El objetivo de estos 90 minutos

Al terminar deberíais haber probado un desarrollo completo —incluyendo un cambio de requisito— guiado exclusivamente por la especificación.

Min	Bloque	Contenido
0–5	Arranque	Comprobación del entorno (ya instalado en casa)
5–12	Marco	SDD en cinco minutos y el caso PenaLib
12–25	A	init y constitución
25–45	A	specify, lectura crítica y clarify
45–58	B	plan, tasks y analyze
58–72	B	implement de la primera versión
72–85	C	El cambio de requisito
85–90	Cierre	Puesta en común, límites y críticas

Los bloques A, B y C son trabajo vuestro. Yo hablo poco (esto, en realidad, no es verdad).

Cómo funciona el taller

Morado: trabajo vuestro

Ejercicios, muchos en pareja. Es el contenido real de la sesión.

Verde: verificación

No sigáis hasta que se cumpla.

Naranja: cuidado

Sitios donde la gente se atasca.

Azul: contexto

Explicación, no exige acción.

Todo está también en el manual, con más detalle. Si os perdéis, repasadlo después.

Arranque: comprobación del entorno

min. 0–5

Levantad la mano si alguna de estas cinco falla:

1. `python3 --version` responde (3.11 o superior).
2. `git --version` responde.
3. `uv --version` responde (es el gestor de paquetes de Astral).
4. `claude --version` responde.
5. `specify --help` responde.

Esto tenía que venir hecho de casa

La guía de instalación previa se envió con la convocatoria. Hoy sólo comprobamos. Si algo falla, levantad la mano: siempre se puede trabajar en pareja.

Recordatorio: Claude Code contra OpenRouter

Las cuatro líneas que deberíais tener ya en vuestro `~/.bashrc` o `~/.zshrc`:

```
export OPENROUTER_API_KEY="sk-or-..."
export ANTHROPIC_BASE_URL="https://openrouter.ai/api"
export ANTHROPIC_AUTH_TOKEN="$OPENROUTER_API_KEY"
export ANTHROPIC_API_KEY="" # imprescindible: vacía
```

Tres avisos

La 1.^a línea lleva *vuestra* clave de OpenRouter, la que preparasteis antes del taller.

La 4.^a tiene que quedar vacía; si no, Claude intenta autenticarse contra Anthropic.

Vuestra clave es dinero y es personal: no debe acabar en el repositorio.

Fichero de apoyo: `bashrc-openrouter.txt` en tinyurl.com/sdd-edd

Comprobación de extremo a extremo

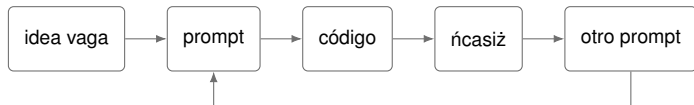
1. `specify self check` responde y os dice la última versión disponible.
2. Ejecutad `claude`.
3. Dentro de Claude, `/status` muestra `ANTHROPIC_AUTH_TOKEN` y la URL de OpenRouter (no una cuenta de Anthropic).
4. Vuestra primera petición aparece en el panel de actividad de OpenRouter. Ahí veréis el gasto durante toda la sesión.
5. Salid de Claude con `/exit`.

Checkpoint 0

`/status` apunta a OpenRouter, `specify` responde y tenéis saldo. Si esto funciona, lo demás va a funcionar.

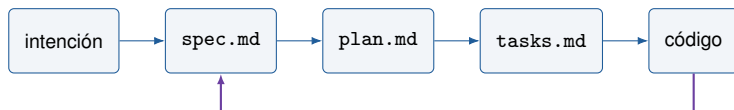
SDD en cinco minutos, y el caso

min. 5–12



- Las decisiones de negocio acaban enterradas en el código.
- La conversación con el agente se pierde.
- Nadie puede revisar *qué se decidió*, sólo *qué se escribió*.

El bucle de hoy



el cambio vuelve aquí, no al código

- Cada fase produce un documento que la siguiente consume.
- Todo son ficheros Markdown versionados, revisables en un *pull request*.
- El código deja de ser la fuente de la verdad.

Lo que SDD no es

- **No es documentación.** La documentación describe lo que ya se hizo; la especificación es la *entrada* del proceso.
- **No es magia.** No elimina trabajo: lo mueve de sitio.
- **No es determinista.** Dos personas con el mismo enunciado obtendrán *specs* distintas. Lo veremos hoy, en directo.
- **No es gratis.** Consume tokens y tiempo. Al final de la sesión tendréis cifras reales de vuestro propio gasto.

El encargo, tal y como llega

La biblioteca del centro presta libros a sus socios. Cada prestamo tiene una fecha de salida y una fecha de devolucion prevista. Cuando un socio devuelve un libro mas tarde de lo previsto se genera una penalizacion. Un socio que tenga penalizaciones pendientes no puede llevarse mas libros. Necesitamos una herramienta que registre prestamos y devoluciones y que calcule las penalizaciones.

Cuatro líneas. Esto es exactamente lo que os diría la persona responsable de la biblioteca... y lo que mucha gente le pasaría a un agente antes de decir nhazme la aplicacónz.

Pregunta para la sala

¿Cuántas decisiones de negocio quedan sin resolver en ese enunciado?

(15 segundos. Decid un número en voz alta.)

Diez, como mínimo

1. ¿Días naturales o hábiles? ¿Festivos?
2. ¿Cuánto se penaliza?
3. ¿Hay tope máximo?
4. ¿Qué es rpendientež?
5. ¿Se puede renovar?
6. ¿Cuántos libros a la vez?
7. ¿Devolver el día del vencimiento cuenta?
8. ¿Y si el libro se pierde?
9. ¿Todos los socios son iguales?
10. ¿Quién anula una penalización? ¿Queda registro?

El problema no es que falten

Un agente sin especificación responderá a las diez por su cuenta, en silencio y de forma plausible... y os enteraréis cuando el sistema esté en producción.

Bloque A: del proyecto vacío a la especificación

min. 12–45

Bloque A: qué vamos a hacer

1. Inicializar el proyecto y el repositorio.
2. Escribir la constitución. (Con trampa.)
3. Generar la especificación a partir del enunciado de la biblioteca.
4. Leerla críticamente. **Aquí está el 60 % del valor de la sesión.**
5. Resolver las ambigüedades con `/speckit-clarify`.

```
specify init penalib --integration claude
cd penalib

# Spec Kit ya no crea el repositorio: hay que hacerlo a mano
git init
specify extension add git

# primer commit, para tener contra que comparar en el bloque C
git add -A
git commit -m "chore: andamiaje de Spec Kit"

git status
claude
```

Fichero de apoyo: `init.txt` en tinyurl.com/sdd-edd

El repositorio no viene hecho

Cambió en la v0.10.0

Hasta esa versión, `specify init` inicializaba el repositorio. Ahora la extensión de `git` es opcional y hay que pedirla: un `git status` justo después de `init` responde que ahí no hay ningún repositorio.

- Sin ese `git init` os faltará media sesión: el ejercicio del *diff* del bloque C no se podrá hacer.
- La extensión también trae `/speckit-git-initialize`, que hace lo mismo desde el agente.

De paso, fijaos: un tutorial escrito contra una versión anterior deja de funcionar en el primer comando. Por eso fijamos la versión a 1.0.2.

Tres atajos de Claude Code para hoy

`Esc` Interrumpe al agente a mitad de una respuesta. Lo usaréis en el bloque B.

`/clear` Vacía el contexto de la conversación. Conviene entre fases largas: los artefactos están en ficheros, no perdéis nada.

`/help` Lista los comandos disponibles, incluidos los del proyecto.

Checkpoint A.1

Existen `.specify/` y `.claude/skills/`; `git status` responde como repositorio; `git log` muestra vuestro commit inicial; `/speckit-` autocompleta.

La constitución

- No es un manifiesto: es un conjunto de *restricciones* que el agente leerá cada vez que planifique o implemente.
- Se guarda en `.specify/memory/constitution.md`.
- Se puede editar a mano. Es Markdown normal.

Corta gana a larga

Con cinco principios bien elegidos, el agente los respeta. Con veinte, el contexto se diluye y empieza a ignorarlos selectivamente. Igual que con las guías de estilo humanas.

Prompt de la constitución

```
/speckit-constitution Crea los principios de un proyecto
pequeno de linea de comandos en Python mantenido por
estudiantes. Prioridades: (1) toda regla de negocio debe
estar cubierta por al menos una prueba automatica con un
caso limite; (2) el calculo de importes y fechas debe ser
determinista y no depender de la zona horaria del equipo;
(3) sin dependencias externas salvo el marco de pruebas;
(4) todo cambio de comportamiento visible obliga a
actualizar la especificacion antes que el codigo;
(5) el codigo debe ser limpio y legible.
```

Fichero de apoyo: `constitution.es.txt` en tinyurl.com/sdd-edd

Abrid después `.specify/memory/constitution.md` y leedlo.

El punto y el guion

Vais a tropezar con esto

La documentación oficial escribe `/speckit.constitution`, con punto. La integración de Claude instala *skills*, cuyo separador es el guion: en Claude Code se escribe `/speckit-constitution`.

- Otros agentes usan `$speckit-constitution` o `/skill:speckit-constitution`.
- Algunos mensajes de error del propio Spec Kit todavía imprimen la forma con punto. Es un defecto conocido: traducid mentalmente.
- Los ficheros están en `.claude/skills/`. Se pueden abrir y leer.

Ejercicio A.1

Principios verificables · 4 minutos, en pareja

Hemos colado un principio tramposo a propósito.

Los cuatro primeros se pueden comprobar: o hay una prueba con un caso límite, o no la hay. El quinto (¡el código debe ser limpio y legible!) no se puede comprobar y por tanto no restringe nada.

Reescribid el principio 5 para convertirlo en algo verificable, o eliminadlo. Editad el fichero a mano.

Puesta en común rápida: ¿qué habéis escrito?

Describimos *qué* queremos. Ni lenguaje, ni base de datos, ni ficheros.

```
/speckit-specify La biblioteca del centro presta libros a  
sus socios. Cada prestamo tiene una fecha de salida y una  
fecha de devolucion prevista. Cuando un socio devuelve un  
libro mas tarde de lo previsto se genera una penalizacion.  
Un socio que tenga penalizaciones pendientes no puede  
llevarse mas libros. Necesitamos una herramienta que  
registre prestamos y devoluciones y que calcule las  
penalizaciones.
```

Fichero de apoyo: `specify.es.txt` en tinyurl.com/sdd-edd

Van a aparecer más ficheros de los que esperáis

Fichero	Lo crea	Qué contiene
spec.md	specify	el qué y el porqué
checklists/requirements.md	specify, clarify	calidad de los requisitos; implement la mira como puerta
plan.md	plan	pila, arquitectura, decisiones
research.md	plan	alternativas consideradas y descartadas
data-model.md	plan	entidades, campos, validaciones
contracts/	plan	interfaces: firmas, comandos, esquemas
quickstart.md	plan	cómo arrancar y validar a mano
tasks.md	tasks	tareas ordenadas por dependencias

No es ruido: cada fase deja por escrito su propio razonamiento para poder revisarlo por separado.

Regla para orientarse en el zoo de ficheros

Mirad quién lo escribió y en qué fase.

- Lo escribió `specify` → es una decisión de negocio, y no debería contener tecnología.
- Lo escribió `plan` → es una decisión técnica.
- Todos son Markdown y todos se editan a mano. Si algo está mal, se corrige en el fichero que lo origina.
- El conjunto exacto varía según la funcionalidad y la versión. No os alarméis.

Qué acaba de pasar

- Un directorio numerado en `specs/`, con `spec.md`.
- Junto a él, `checklists/requirements.md`: la lista de verificación integrada de calidad de los requisitos.
- Si añadisteis la extensión de git, también una rama para la funcionalidad.

*Ahora empieza el trabajo de verdad.
Y no lo hace el agente.*

Ejercicio A.2 · Lectura crítica

8 minutos, en pareja · anotad las respuestas

1. ¿Qué decisiones de negocio ha tomado el agente que vosotros no habíais tomado?
2. ¿Ha dejado marcas explícitas de ambigüedad? ¿Dónde?
3. ¿Hay algún requisito que no sabríais cómo verificar?
4. ¿Hay algún criterio de aceptación sin números?
5. ¿Se ha colado alguna decisión técnica que no pinta nada aquí?

Guardad las respuestas: las usaréis en el bloque siguiente. Las otras cinco preguntas de la lista (funcionalidad inventada, casos límite que faltan, legibilidad para negocio, contradicciones con la constitución, continuidad del proyecto) están en el manual, sección 5.3, para después.

Ejercicio A.3 · Comparad con la pareja de al lado

2 minutos

Habéis partido del mismo enunciado. Localizad **una** regla de negocio en la que vuestro `spec.md` y el del grupo vecino digan cosas distintas.

Qué acabáis de demostrar

Esa divergencia no es un defecto de la herramienta: es la prueba experimental de que un enunciado ambiguo no determina un sistema.

La diferencia entre hacer esto y no hacer nada es que aquí la divergencia está escrita en un fichero que se puede revisar, en vez de enterrada en un `if` de la línea 240.

Resolved las ambigüedades

/speckit-clarify

- Hace hasta cinco preguntas por pasada, sobre las zonas poco especificadas.
- Se puede ejecutar varias veces, cada una centrada en un área distinta.
- Es lo más parecido a una entrevista de requisitos que vais a tener hoy.

Responded con criterio

Cada respuesta se incorpora a la especificación y condicionará el plan, las tareas y el código. No contestéis lo primero que se os ocurra.

Decisiones para el taller

Decisión	Valor
Unidad de retraso	Días naturales completos, sin contar festivos
Importe	0,20 € por día de retraso y libro
Tope máximo	20,00 € por préstamo
Penalización pendiente	La que no ha sido abonada ni condonada
Límite de préstamos	3 libros simultáneos por socio
Renovaciones	Una de 14 días, sólo sin penalizaciones pendientes
Devolución el día del vencimiento	No es retraso, sea la hora que sea

Podéis usar las vuestras si las argumentáis. Después, comparad el `spec.md` con las respuestas del ejercicio A.2.

Cierre del bloque A

Checkpoint A.2 · antes de pasar al plan

- No queda ninguna marca de ambigüedad pendiente en `spec.md`.
- Todos los importes, plazos y límites tienen un número concreto.
- La especificación sigue sin mencionar lenguaje, ficheros ni librerías.
- Habéis hecho commit.

La especificación es código fuente. Trátadla como tal.

Bloque B: plan, tareas e implementación

min. 45–72

Ahora, y sólo ahora, se decide la tecnología. Todo el grupo, la misma pila:

```
/speckit-plan Aplicacion de linea de comandos en Python 3.11  
usando exclusivamente la biblioteca estandar, mas pytest.  
La persistencia es un unico fichero JSON en el directorio de  
trabajo. Sin base de datos, sin servidor, sin interfaz web.  
Las fechas se manejan con date, nunca con datetime. La logica  
de calculo de penalizaciones debe estar en un modulo puro,  
sin entrada/salida, para poder probarla de forma aislada.
```

Fichero de apoyo: `plan.es.txt` en tinyurl.com/sdd-edd

El ciclo completo

Fase	Comando	Produce
Principios	<code>/speckit-constitution</code>	<code>constitution.md</code>
Qué y por qué	<code>/speckit-specify</code>	<code>spec.md</code> + lista de requisitos
Ambigüedades	<code>/speckit-clarify</code>	<code>spec.md</code> sin huecos
Cómo	<code>/speckit-plan</code>	<code>plan.md</code> y artefactos de diseño
Calidad	<code>/speckit-checklist</code>	listas de verificación
Orden	<code>/speckit-tasks</code>	<code>tasks.md</code>
Coherencia	<code>/speckit-analyze</code>	informe (sólo lectura)
Construir	<code>/speckit-implement</code>	código y pruebas
Cerrar	<code>/speckit-converge</code>	tareas que faltaban

Opcional no quiere decir prescindible



Lo que separa SDD de una charla ordenada con un agente

`clarify`, `checklist` y `analyze` son justo los comandos que casi nadie usa cuando prueba la herramienta por su cuenta. Son los que más peso tienen hoy.

Cada artefacto tiene un lector distinto

Artefacto	Responde a	Quién debería poder revisarlo
constitution.md	£Bajo qué reglas trabajamos?	El equipo, una vez
spec.md	£Qué debe hacer y por qué?	Alguien de negocio, sin perfil técnico
plan.md	£Con qué tecnología?	Una persona técnica
tasks.md	£En qué orden?	Quien implemente, humano o agente
Código	—	Se revisa, pero ya no manda

Si vuestro `spec.md` habla de índices de base de datos, está mal escrito.

Cuánto os pregunta: dos cosas distintas

Preguntas de requisitos

Las decide Spec Kit, y las controláis vosotros eligiendo recorrido:

- **Corto:** `specify` → `plan` → `tasks` → `implement`. Casi no pregunta.
- **Completo:** + `clarify`, `checklist`, `analyze`. `clarify` hace hasta 5 preguntas por pasada.

Permisos

Los decide Claude Code. `Shift+Tab` para cambiar:

- **Manual:** pregunta antes de casi todo.
- **Aceptar ediciones:** edita sin preguntar.
- **Plan:** explora, no modifica.
- **Auto:** otro modelo revisa por vosotros.
- **Saltarse permisos:** sin comprobaciones.

Hoy hacemos el recorrido completo. Las preguntas son el contenido.

Qué modo usar hoy

Recomendación

Manual mientras generáis artefactos (bloque A y plan/tareas).

Aceptar ediciones mientras implementáis.

- El modo automático es razonable cuando ya sabéis qué estáis construyendo y el error es barato.
- Es mala idea cuando el objetivo es aprender qué está decidiendo el agente.
- Regla general fuera del aula: cuanto menos reversible sea la acción, menos automatismo. Editar un fichero versionado es reversible; ejecutar órdenes contra un sistema real, no.

Qué revisar en el plan

Se generan varios ficheros, no sólo `plan.md`: `research.md`, `data-model.md`, `contracts/`, `quickstart.md`. Abridlos por encima.

Después, dos preguntas:

1. ¿El plan respeta la constitución? Si prohibisteis dependencias externas, no debería aparecer ninguna.
2. ¿El plan cubre todos los requisitos de la especificación, o se ha dejado alguno por el camino?

Tareas y revisión cruzada

`/speckit-tasks`

Lista ordenada por dependencias, agrupada en fases. Mirad si las pruebas aparecen antes o después de cada regla: es un buen indicador de si vuestra constitución está teniendo efecto real.

`/speckit-analyze`

Análisis de coherencia y cobertura entre `spec.md`, `plan.md` y `tasks.md`. Es de sólo lectura: no toca nada, produce un informe.

Ejercicio B.1 · El hallazgo

4 minutos

Anotad un problema concreto detectado por analyze y clasifícalo: ¿es un fallo de la especificación, del plan o de las tareas?

Corregidlo **en el artefacto donde nace** el problema, no en el que lo manifiesta.

Arreglar el origen, no el síntoma.

Este reflejo es la competencia central de la sesión.

No pidáis la aplicación entera

Aquí implementamos sólo el módulo de cálculo de penalizaciones y sus pruebas. Si dejáis al agente suelto, pasará veinte minutos generando código que no vamos a mirar y consumirá vuestro saldo.

Si se desmanda: Esc y reformular. Interrumpir es barato; deshacer, no.

Buen momento para pasar en Claude a *aceptar ediciones* con **Shift+Tab** y evitar que os pregunte treinta veces.

Antes de implementar: consolidad en git

```
/speckit-git-commit
```

El propio agente os lo ofrecerá: la extensión de git registra un *hook* en el evento `before_implement`.

Por qué justo aquí

Es el último punto del ciclo en el que todo lo que hay en el repositorio son decisiones: especificación, plan y tareas, sin una línea de código.

Si confirmáis ahora, el commit siguiente contendrá exactamente lo que produjo la implementación, y el historial se lee como *¿esto decidimos / esto se construyó a partir de ello?*. Si no, ambas cosas se mezclan y perdéis esa separación... que es justo la que necesitáis en el ejercicio C.1.

Implementar

```
/speckit-implement Implementa unicamente las tareas relativas  
al modulo puro de calculo de penalizaciones y sus pruebas.  
No implementes todavia la persistencia ni la interfaz de  
linea de comandos.
```

Fichero de apoyo: `implement.es.txt` en tinyurl.com/sdd-edd

B.2 · Hoja de observación, mientras el agente trabaja

- ¿En qué tarea se ha atascado o ha tenido que rehacer algo?
- ¿Se nota que ha leído la constitución? ¿En qué, concretamente?
- ¿Ha introducido alguna regla que no está en la especificación?
- ¿Las pruebas cubren los casos límite del ejercicio A.2?

Ejercicio B.3 · Cuánto ha costado esto

1 minuto

Abrid el panel de actividad de OpenRouter y anotad el coste acumulado de la sesión y el de la fase de implementación por separado.

Lo compararemos con el coste de absorber un cambio de requisito, y en la discusión final tendréis cifras reales en lugar de intuiciones.

Después, las pruebas:

```
python3 -m pytest -q
```

Cierre del bloque B

Checkpoint B.1

Las pruebas pasan, existe un módulo de cálculo sin entrada/salida, y podéis señalar en el código la línea que implementa el tope de 20 € y la que implementa el importe de 0,20 €.

Si no podéis señalarlas, no habéis revisado el código. Hacedlo ahora: lo vais a necesitar en el bloque siguiente.

Bloque C: el cambio de requisito

min. 72–85

Sin este bloque, lo anterior es sólo una forma elaborada de pedir código

Ahora cambian las reglas.

A partir del proximo curso distinguimos dos tipos de socio. Los estudiantes tienen tres dias de gracia antes de que empiece a contar la penalizacion y un tope maximo de 10 euros. El profesorado no tiene penalizaciones, pero no puede tener mas de cinco prestamos a la vez. Ademias, el personal de biblioteca debe poder condonar una penalizacion indicando obligatoriamente un motivo, y debe quedar registro de quien la condono y cuando.

Fichero de apoyo: `specify2.es.txt` en tinyurl.com/sdd-edd

Lo que este cambio rompe

- Un atributo nuevo en el modelo: el tipo de socio.
- Dos reglas de cálculo divergentes donde antes había una.
- Un límite que depende del tipo de socio.
- Una operación de auditoría que antes no existía.

En un desarrollo convencional esto son varias horas de tocar código y descubrir efectos colaterales.

Primera decisión: ¿dónde vive la enmienda?

Opción	Cuándo tiene sentido	Qué cuesta
A. Enmendar 001	El cambio redefine reglas ya especificadas: importe, tope, quién penaliza. La spec debe describir el sistema tal y como es hoy.	Spec única y legible, pero el ¿qué decidimos y cuándo sólo vive en el historial de git.
B. Crear 002	El cambio añade una capacidad separable, con su propia entrega: p. ej. la condonación auditada.	Dos specs hablan del mismo dominio y pueden contradecirse. Hay que dejar escrito cuál manda.
C. Mixta	Lo que haría un equipo real: enmendar 001 y abrir 002 para la condonación.	Más ceremonia y dos ramas que coordinar.

Un detalle que sorprende a todo el mundo

Cambiar de rama no cambia de funcionalidad

Spec Kit no deduce la funcionalidad activa de la rama de git: la lee de `.specify/feature.json` (o de `SPECIFY_FEATURE_DIRECTORY`).

Un `git checkout` por sí solo no cambia el objetivo de los comandos.

Para el taller: opción A, enmendar 001.

Es la que hace posible el ejercicio del *diff*.

El procedimiento correcto

Fijaos en lo que *no* vamos a hacer: no vamos a abrir el módulo de cálculo.

1. `/speckit-specify` — actualizar la especificación. Es donde nace el cambio.
2. `/speckit-clarify` — ¿los tres días de gracia cuentan para el límite de préstamos o sólo para el importe? ¿Qué pasa con las penalizaciones ya generadas de quien ahora es profesorado?
3. `/speckit-plan` y `/speckit-tasks` — regenerar.
4. `/speckit-analyze` — comprobar coherencia.
5. `/speckit-implement` — acotando otra vez al módulo de cálculo.

Ejercicio C.1 · El diff que importa

```
git diff --stat HEAD~1  
git diff HEAD~1 -- specs/
```

6 minutos · una frase por pregunta

- ¿Cuántas líneas han cambiado en *specs/* y cuántas en el código?
- Leyendo sólo el *diff* de la especificación, ¿queda claro qué se ha decidido y por qué?
- Si dentro de un año alguien pregunta ¿por qué el profesorado no tiene multas?, ¿dónde está la respuesta?

El punto que hay que llevarse a casa

Flujo tradicional

- El cambio se aplica al código.
- La documentación se desfasa en silencio.
- La razón de la decisión se pierde.

Flujo de hoy

- El cambio se aplica al documento.
- El código se deriva de él.
- La razón queda en un *diff* revisable.

*No es magia ni elimina trabajo:
cambia dónde vive la verdad del sistema.*

Cierre y discusión

min. 85–90

1. ¿En qué tipo de proyecto *no* compensa este proceso? Sed concretos.
2. La especificación también envejece y también acumula deuda. ¿Qué evita que `spec.md` se convierta en el nuevo legado ilegible?
3. ¿Quién debería escribir la especificación en un equipo real: quien desarrolla, quien define el producto, ambos?
4. ¿Cómo revisaríais una especificación en un *pull request*? ¿Qué buscaríais?
5. Mirad el panel de OpenRouter. Con esas cifras delante, ¿cuándo sale a cuenta este proceso y cuándo no?
6. Si el modelo genera una especificación plausible pero equivocada, ¿qué mecanismo lo detecta antes de llegar a producción?

Las críticas que hay que tomarse en serio

- **Coste.** El ciclo completo consume muchos más tokens que pedir código directamente. Para un script de veinte líneas es absurdo.
- **Ceremonia.** Hay proyectos donde el trabajo de mantener artefactos supera el valor que aportan.
- **Falsa sensación de rigor.** Una especificación bien redactada puede estar completamente equivocada. El formato no valida el contenido.
- **Nueva deuda.** Cuarenta *specs* desactualizadas son tan malas como cuarenta módulos sin documentar.
- **Dependencia** de una herramienta joven que rompe su interfaz cada pocas versiones. Lo habéis sufrido hoy.

Qué llevaros, aunque no volváis a usar Spec Kit

1. Un enunciado ambiguo no determina un sistema. Alguien decidirá lo que falta: mejor que sea explícito.
2. Si un requisito no tiene número, todavía no es un requisito.
3. El sitio donde se arregla un problema es donde nace, no donde se manifiesta.
4. Separad el *qué* del *cómo*, y dad a cada uno el revisor que le corresponde.
5. Ante un cambio, id al artefacto de más arriba que esté afectado y regenerad hacia abajo.

Para llevaros el trabajo a casa

El proyecto `penalib` es vuestro. Merece la pena que contenga:

1. `constitution.md`, con el principio 5 corregido.
2. `specs/<NNN>-.../spec.md` en su versión final, tras el cambio.
3. `plan.md` y `tasks.md`.
4. El módulo de cálculo y sus pruebas, con `pytest` en verde.

Antes de compartir el repositorio con nadie

Comprobad que vuestra clave de OpenRouter no viaja dentro:

```
git log -p | grep sk-or-
```

Si aparece, revocadla desde el panel de OpenRouter.

Para seguir

- Repositorio: `github.com/github/spec-kit`
- Documentación: `github.github.io/spec-kit`
- Metodología completa: el fichero `spec-driven.md` del repositorio
- Claude Code: `code.claude.com/docs`
- Personalización: extensiones, presets y bundles (`specify extension`, `specify preset`, `specify bundle`)
- Proyectos existentes: el flujo *brownfield* y `/speckit-converge`

Al buscar tutoriales

Casi todo el material anterior a mediados de 2026 usa la sintaxis antigua y da por hecho que `specify init` crea el repositorio. Contrastad siempre con la documentación de vuestra versión.

Gracias

Emilio Delgado Muñoz

Institutos Universitarios de Tecnologías Informáticas Aplicadas
Universidad de Extremadura · edm@unex.es

Material adaptado del taller original de Gregorio Robles (URJC),
gsyc.urjc.es/grex/sdd

Transparencias de reserva

Errores frecuentes (1/2)

Síntoma	Qué hacer
<code>git status</code> dice que no es un repositorio	Normal desde la v0.10.0. <code>git init</code> y <code>specify extension add git</code> .
Claude Code no reconoce <code>/speckit-...</code>	O arrancasteis fuera de la carpeta del proyecto, o escribís punto en vez de guion.
Un mensaje os manda ejecutar <code>/speckit.algo</code>	Defecto conocido: traducid el punto por un guion.
<code>command not found: claude</code>	No está en el PATH. Terminal nueva y <code>claude doctor</code> .
Errores de autenticación	<code>/status</code> : casi siempre es <code>ANTHROPIC_API_KEY</code> sin vaciar.

Errores frecuentes (2/2)

Síntoma	Qué hacer
Aparecen ficheros inesperados en <code>specs/</code>	Es normal: cada fase escribe sus artefactos.
La spec sale llena de detalles técnicos	Vuestro prompt los incluía. Rehacedla hablando sólo de comportamiento.
La spec es enorme y genérica	Demasiado alcance. Una funcionalidad por especificación.
<code>implement</code> se eterniza	<code>Esc</code> y relanzar indicando tareas concretas.
El código no se parece al plan	<code>/speckit-converge</code> : compara y añade lo que falta.
El agente ignora la constitución	Principios no verificables o demasiados. Recortadla.
Tu compañero obtiene otro resultado	Es lo esperable. Por eso se revisan los artefactos.

Referencia: línea de comandos

```
# instalacion fijando version
uv tool install specify-cli \
  --from git+https://github.com/github/spec-kit.git@v1.0.2

# nuevo proyecto
specify init <nombre> --integration claude
specify init --here --force --integration claude

# el repositorio NO se crea solo desde la v0.10.0
git init
specify extension add git

specify integration list
specify self check
specify self upgrade [--dry-run] [--tag vX.Y.Z]
```

Referencia: dentro del agente

/speckit-constitution	principios del proyecto
/speckit-specify	que construir y por que
/speckit-clarify	resolver ambigüedades (antes de plan)
/speckit-plan	como construirlo
/speckit-checklist	calidad de los requisitos
/speckit-tasks	en que orden
/speckit-analyze	coherencia (antes de implement)
/speckit-implement	construir
/speckit-converge	reconciliar código y artefactos
/speckit-git-commit	confirmar; se ofrece antes de implement
/speckit-git-feature	crear la rama de la funcionalidad
/speckit-git-initialize	inicializar el repositorio

Referencia: Claude Code

```
claude          # abrir sesion en el directorio actual
claude --version
claude doctor    # diagnostico de instalacion

/status         a que endpoint y con que token se conecta
/help          comandos disponibles
/clear         vaciar el contexto
/logout        borrar una sesion de Anthropic guardada
/exit          salir
Esc            interrumpir al agente
Shift+Tab      cambiar de modo de permisos
```

Reglas de oro

- La especificación describe comportamiento observable; el plan describe tecnología. No los mezcléis.
- Una funcionalidad, una carpeta en `specs/`.
- `clarify` antes de `plan`; `analyze` antes de `implement`.
- Si un requisito no tiene número, todavía no es un requisito.
- Ante un cambio, editad el artefacto de más arriba que esté afectado y regenerad hacia abajo.
- Commit de los artefactos por separado del código.
- Modo manual mientras generáis artefactos; aceptar ediciones mientras implementáis.

Si usáis otro agente

Agente	Invocación
Claude Code	<i>Skills</i> con guion: <code>/speckit-specify</code> , en <code>.claude/skills/</code>
Agentes de comandos	Con punto: <code>/speckit.specify</code> , como la documentación oficial
GitHub Copilot (editor)	Instala <code>skills</code> por defecto. Para comandos, inicializar con <code>--integration-options</code>
Copilot CLI	Selección con <code>/agents</code>
Codex CLI, ZCode	<code>\$speckit-specify</code>
Kimi y similares	<code>/skill:speckit-specify</code>

`specify integration list` manda sobre cualquier tutorial.

El método, los prompts y los artefactos son idénticos en todos.